

3.3 AI Evaluation

Tài liệu này tách rõ hai loại số: **số đã đo được trong repo** (kiểm chứng lại được bằng một lệnh) và **số quan sát trên ticket thật** (cỡ mẫu nhỏ, ghi rõ là quan sát chứ không phải benchmark). Không có số nào ở đây là ước lượng suy ra từ giả định.

Evaluation Methodology

Không dùng benchmark ngoài. Cách đánh giá bám vào chính chỗ mà một AI solution kiểu này hay hỏng: **output trông hợp lý nhưng không chạy được**. Nên tiêu chí trung tâm không phải "output có giống test case người viết không" mà **"output có chạy được trên hệ thống thật không"**.

Bốn tầng, chạy theo thứ tự, tầng sau chỉ chạy nếu tầng trước qua:

Tầng	Đánh giá gì	Cơ chế	Tự động
E1 — Structural	Output có đúng shape backend parse không	JSON schema pin trong prompt của caller; parse fail = run fail	✓
E2 — Grounding	Spec có bịa selector / URL / TODO không	Placeholder gate pre-flight, verdict passed / blocked / rejected	✓
E3 — Executable	Spec có thật sự chạy green không	Chạy spec trên app thật; execution-analyzer gán failure class	✓
E4 — Human	Case có đủ, đúng nghiệp vụ, đáng automation không	Review Center — người duyệt từng case	✗ (cố ý)

Điểm đáng chú ý: **E2 và E3 là evaluator chạy trong production, không phải test harness rời**. Mọi run đều đi qua chúng, nên chất lượng được đo liên tục chứ không chỉ ở lúc đánh giá.

Evaluation Scenarios

ID	Scenario	Kiểm chứng
S1	Build knowledge base trên một repo chưa từng index	Đạt indexed, knowledge.json có route và selector khớp source
S2	Sinh test case cho ticket có AC rõ ràng	Mỗi AC có ít nhất một case map tới
S3	Sinh test case cho ticket có AC mơ hồ	requirement-analyst nêu ra điểm mơ hồ thay vì tự bịa
S4	Live-harness trên app đã authenticated	Đi hết step, emit spec, spec chạy green ở lần đầu
S5	Live-harness khi test data chưa có	Agent tạo data qua UI và bake setup vào spec
S6	Spec cố tình chứa selector bịa	Placeholder gate ra blocked / rejected trước khi execution
S7	UI đổi selector sau khi spec đã sinh	execution-analyzer gán test_defect, heal sửa page object, spec xanh lại

S8	App thật có bug	Gán product_defect, không heal cho xanh
S9	Knowledge base stale	Trạng thái hiển thị đúng; entry verified-at-runtime không bị source-inferred ghi đè
S10	Credential hết hạn giữa run	Status refreshable, CLI renew, PUT /credentials/claude/refreshed ghi ngược, run tiếp tục
S11	Vượt cost ceiling	Run dừng có lý do, không chạy tiếp âm thầm
S12	App sau SSO/MFA	Chạy qua Local Agent; cookie và storageState không lên server

S6, S8, S10, S11 là **negative scenario** — chúng đánh giá guardrail, và guardrail hỏng nguy hiểm hơn model kém.

Test Dataset

Chưa có eval dataset đóng gói. Đánh giá chạy trên **ticket thật** của hai project nội bộ (EmeHub và Q-Agent tự test lẫn nhau) cộng bộ test tự động trong repo.

Nguồn	Quy mô	Tính chất
Test backend EmeHub	788 hàm test	Deterministic, chạy mỗi PR
Test backend Q-Agent	1358 hàm test	Deterministic
Ticket thật dùng để quan sát	Cỡ mẫu nhỏ, web UI, 3–12 case/ticket	Không phải benchmark

Đây là **giới hạn đã biết**, ghi ở phần Limitations.

Evaluation Metrics

Metric	Định nghĩa	Đo ở đâu
M1 — AC coverage	% acceptance criteria có ≥ 1 test case map tới	Cột mapping AC $\rightarrow$ case ở Review Center
M2 — Review acceptance rate	% case được reviewer approve mà không sửa	Review Center
M3 — Grounding pass rate	% spec qua placeholder gate ở lần đầu (passed)	Placeholder gate
M4 — First-run green rate	% spec chạy green ngay lần đầu, không heal	Execution
M5 — Heal convergence	% spec đỏ được sửa xanh trong ≤ 3 lần	Execution
M6 — Failure classification correctness	% failure class mà reviewer đồng ý	Đối chiếu thủ công
M7 — Cost / ticket	USD cho một run đầy đủ	claude_usage, attribute theo owner
M8 — Wall-clock / ticket	Thời gian từ Create run tới Publish	Run timeline
M9 — Người-phút / ticket	Thời gian người thực sự ngồi thao tác	Đo thủ công

M4 là metric quan trọng nhất, vì nó chính là phát biểu sản phẩm: *spec sinh từ DOM thật chạy được ngay, không cần heal pass*.

Results

Đã đo được trong repo — kiểm chứng lại được

Kết quả	Số	Kiểm chứng bằng
Test backend EmeHub xanh	788 hàm test	cd api && uv run pytest
Test backend Q-Agent xanh	1358 hàm test	tương tự ở repo q-agent
Typecheck + build frontend xanh	—	npm run typecheck && npm run build
Contrast audit light mode	241 element fail → 2 (cả hai là logotype Jira #fff trên #2684ff, WCAG miễn trừ)	Audit script trên container build
Kiểm thử UI trên container build	9 route × 2 mode × 4 accent = 72 lượt load, 0 console error	Playwright
API router	14	ls api/app/routers/*.py
View chạy trên endpoint thật	11	—
Skill chuyên biệt	14 (Q-Agent) + 1 (hub)	find skills -name SKILL.md
ADR đã accept	12	ls docs/adr/*.md

Quan sát trên ticket thật — cỡ mẫu nhỏ

Các mục dưới đây là **quan sát định tính** trên số ticket đủ ít để không tuyên bố thành tỉ lệ. Đội chủ động không quy chúng thành phần trăm.

Metric	Quan sát
M3 — Grounding pass rate	Spec sinh ở mode live-harness qua placeholder gate; spec sinh ở mode blind trên knowledge base stale là nguồn blocked chủ yếu
M4 — First-run green rate	Ở live-harness, spec chạy green ngay là trường hợp thường gặp — đúng như thiết kế, vì mọi selector trong spec đã được dispatch và verify trên DOM thật trước khi emit
M5 — Heal convergence	Ở mode blind, phần lớn ca đổ do selector hội tụ trong ≤3 lần khi heal có grounding DOM
M6 — Failure classification	product_defect được gán đúng ở các ca app thật lỗi; đây là chỗ cần thêm mẫu để nói chắc
M7 — Cost / ticket	Hiển thị theo từng run ở màn Execution, nên tính được giá cho mỗi ticket

Bảng số cần điền trước khi công bố ra ngoài đội:

Metric	Cỡ mẫu	Kết quả
M1 AC coverage	[N ticket]	[...]
M2 Review acceptance rate	[N case]	[...]
M3 Grounding pass rate	[N spec]	[...]
M4 First-run green rate	[N spec]	[...]

M7 Cost / ticket	[N run]	[... USD]
M8 Wall-clock / ticket	[N run]	[... phút]
M9 Người-phút / ticket	[N ticket]	[... phút]

Failure Cases

Ghi lại những ca đã gặp thật, kèm cách hệ thống phản ứng:

#	Failure case	Nguyên nhân	Hệ thống làm gì	Còn thiếu gì
F1	Spec blind chứa selector không tồn tại	Knowledge base stale sau khi UI đổi	Placeholder gate ra blocked trước execution	Cảnh báo stale chủ động hơn
F2	Live-harness dừng giữa chừng	Test account sai hoặc base URL không reachable	Không emit spec, báo bước fail	Thông báo phân biệt hai nguyên nhân này rõ hơn
F3	click_at_xy pass nhưng spec fail	Toạ độ trùng <a> con, spec click container không handle click	Đã sửa ở tầng skill: dispatch trên chính selector sắp emit, verify hiệu ứng thật	—
F4	Credential quá expiresAt bị đọc thành expired	Access token Claude OAuth chỉ sống vài giờ	Tách refreshable khỏi expired; expired chỉ đến từ CLI bị reject thật	—
F5	Heal làm spec "xanh" bằng cách nói assertion	Model tối ưu cho tín hiệu pass	Heal bounded 3 lần, chỉ chạy khi có grounding DOM; failure class hiển thị để người phản bác	Chưa có checker tự động phát hiện assertion bị nói
F6	Case sinh ra đúng cú pháp nhưng sai nghiệp vụ	Model không có domain context ngoài source	Gate 1 — reviewer từ chối	Đây là lý do gate là bắt buộc, không phải cảnh báo

Limitations

Nói thẳng:

1. **Chưa có eval dataset đóng gói.** Không có bộ ticket cố định + expected output để chạy lại sau mỗi thay đổi prompt. Hệ quả: chưa phát hiện được regression về chất lượng AI một cách tự động — chỉ phát hiện được regression về code (788 + 1358 test).
2. **Cỡ mẫu ticket nhỏ.** Đủ để nói cơ chế hoạt động, không đủ để công bố tỉ lệ.
3. **Chưa có LLM-as-judge.** M6 (failure classification correctness) đang đối chiếu thủ công.
4. **Chưa đo M9 có kiểm soát.** Con số "người-phút/ticket" hiện là quan sát của chính người xây, tức là có bias.
5. **Live-harness cần app đang chạy và tài khoản test.** Không áp dụng được cho app không có môi trường test truy cập được.
6. **Đánh giá tập trung vào web UI.** API test, mobile, desktop chưa nằm trong phạm vi đo.

7. **Agent cutover chưa hoàn tất.** Q-Agent mới tiêu thụ identity từ hub; credential và project vẫn còn bản của agent. Nên phần "khai một lần dùng mọi nơi" đã đúng với identity, chưa đúng toàn phần.

Before / After Comparison

Về cơ chế — kiểm chứng được, không phụ thuộc cỡ mẫu

Chiều	Trước	Sau
Grounding của AI	Model đoán selector từ ngữ cảnh nó nhìn thấy	Selector lấy từ DOM thật, đã dispatch và verify trước khi emit
Thứ tự làm việc	Sinh spec → chạy → fail → heal	Chạy thật → verify → mới sinh spec
Kiểm tra trước khi chạy	Không có	Placeholder gate: passed / blocked / rejected
Đọc kết quả đồ	Người tự đoán test sai hay product sai	Failure class 5 loại, gán tự động, người phản bác được
Chuẩn hoá prompt	Mỗi người tự viết prompt	15 skill đi kèm sản phẩm, review và cải tiến tập trung
Cấu trúc automation	File spec rời, trùng lặp	Page object + component + fixture dùng lại; UI đổi thì sửa một chỗ
Credential	PAT trong .env từng máy; Claude account chung không attribution	PAT không rời hub; Claude credential 2 lớp, usage per-user
Thu hồi quyền	Nhiều nơi	Revoke một session = logout mọi agent
Human oversight	Ngầm định, không cưỡng chế	3 gate chặn thật ở 3 chỗ có hậu quả ra ngoài

Về thời gian — cần số đo, không suy đoán

Công việc	Trước (ước lượng nội bộ)	Sau	Người còn phải làm gì
QA một ticket, đủ 7 bước	~4.5 giờ	[đo M8 + M9]	Duyệt case ở Review Center, duyệt comment
Sửa spec đồ khi UI đổi	~[A] phút/spec	Heal tự động ≤3 lần	Xác nhận kết quả chạy lại
Cấu hình một agent mới vào hệ thống	Khai lại từ đầu	Đọc contract, dùng lại identity + credential + KB	—
Build knowledge cho member thứ hai	~20 phút build	Clone project từ shared namespace	—

Ồ [...] là số đội phải đo trước khi đưa ra ngoài. Để trống có chủ đích còn hơn điền số ước lượng.